Item Drag & Drop Interaction System

Design note for Mandate UI. This document defines the target model for ItemDragRuntime, item interaction policies, ownership boundaries and first implementation scope.

Core rule

ItemDragRuntime does not move items. It carries a user gesture from a source to a target. The target policy decides what the drop means, and the World server remains authoritative for gameplay state.

1. Purpose

Mandate uses one shared item drag runtime for item-oriented UI interactions across Inventory, Equipment, Storage, Shops and Trade. The runtime owns interaction mechanics only. It never owns gameplay rules, item ownership, persistence, pricing, equipment eligibility, trade state or inventory authority.

A drag operation represents: "the player is carrying an item-related subject from one UI source and is attempting an interaction with another UI target." It does not inherently mean "move this ItemInstance from container A to container B."

This distinction allows the same gesture to represent inventory movement, equipping, storage transfer, purchasing, listing creation or trade offering.

2. Core principles

- Server authority remains unchanged. Drag previews are advisory. Every resulting gameplay operation is validated and committed by the World server.
- The runtime never mutates domain state. It may render a ghost, placement preview, highlight or pending visual state, but it never removes an item from a source or inserts one into a destination.
- Sources describe what is being dragged. Targets decide what a drop means.
- The runtime is not grid-specific. Inventory and Storage use cell placement, Equipment uses slot highlights, while Shops and Trade may use whole-panel interaction targets.
- Primary item activation remains separate from drag and drop.

```
Right click / primary action
    -> item.activate

Left drag/drop
    -> ItemDragRuntime
    -> target policy
    -> domain command/workflow
```

3. Drag subjects

Not every draggable representation is an owned ItemInstance. NPC shop offers, player shop offers and trade offers can all be draggable without being directly owned by the current character.

```
type ItemDragSubject =
  | { kind: 'owned-item'; id: string; revision: number }
  | { kind: 'npc-shop-offer'; shopId: string; offerId: string }
  | { kind: 'player-shop-offer'; shopId: string; offerId: string }
  | { kind: 'trade-item'; tradeId: string; offerId: string; side: 'self' };

interface ItemDragPayload {
  sourceId: string;
  subject: ItemDragSubject;
  presentation: ItemPresentation;
}
```

Presentation data is only for visuals such as the ghost, icon, height and tooltip-like preview. It is not authoritative gameplay data.

4. Runtime responsibilities

- one active drag session
- pointer capture
- drag threshold
- click-to-carry and drag-to-carry
- grab offset
- ghost rendering
- pointer movement
- cross-window hit testing
- target selection
- preview lifecycle
- UI scale handling
- Escape cancellation
- right-click cancellation
- blur / resize cancellation
- cleanup and disposal
- interactive-region refresh

The runtime does not know what Inventory, Storage, Trade or Shop commands mean.

5. Source contract

A source resolves a pointer interaction into a drag payload. It decides whether a subject can be dragged, but not what targets may do with it.

```
interface ItemDragSource {
  element: HTMLElement;
  resolve(event: PointerEvent): ItemDragPayload | null;
}
```

Example source	Drag capability
Inventory item	Draggable
Storage item	Draggable
Equipped item	Draggable
NPC shop offer	Draggable
Player shop offer	Draggable for buyer interaction
Opponent trade item	Not draggable
Locked own trade item	Not draggable

6. Target contract

Targets interpret the payload. Runtime treats target-specific preview data as opaque.

```
interface ItemDropTarget<T = unknown> {
  element: HTMLElement;
```

```
  preview(payload: ItemDragPayload, pointer: Point): DropPreview<T>;

  drop(payload: ItemDragPayload, preview: DropPreview<T>): void | Promise<void>;
}
```

A grid target may resolve x, y and page. An Equipment target may resolve an equipment slot. A Shop target may only highlight the Inventory panel and start a purchase workflow.

7. Interaction matrix

Source	Target	Semantic operation
Inventory	Inventory cell	Move/reposition item
Inventory	Equipment slot	Equip
Equipment	Inventory	Unequip
Inventory	Storage cell	Deposit
Storage	Inventory cell	Withdraw to requested position
Storage	Inventory receive area	Withdraw to first fitting position
Storage	Storage cell	Reposition
Inventory	Trade self offer	Add item to trade
Trade self offer	Inventory	Remove item from trade
Trade other offer	Inventory	Reject
Inventory	Player Shop Editor	Begin listing workflow
Own shop listing	Inventory	Remove listing
NPC Shop offer	Inventory	Begin purchase workflow
Player Shop offer	Inventory	Begin purchase workflow

8. Inventory placement versus Inventory receive

Inventory supports two related target semantics.

Explicit placement

The player intentionally dropped onto a specific cell. The operation should preserve that intent and request that exact placement. If the position becomes invalid before the authoritative server commit, the operation should be rejected rather than silently placing the item somewhere unexpected.

```
Storage item -> Inventory cell (x=2, y=4, page=1)
    -> request exact placement
```

Inventory receive

The player requests that an item enter Inventory without specifying an exact placement. The server finds the first valid fitting position.

- right-click shop purchase
- world pickup
- quest reward
- loot reward
- mailbox claim
- Ctrl-click Storage withdrawal
- drop a shop offer onto the general Inventory receive area

```
requested placement exists
    -> validate exact requested position

no requested placement
    -> find first valid fitting position
```

The placement algorithm must remain authoritative on the server.

9. Shared receiving rules

InventoryReceiveTarget is a UI policy, but the underlying receive behavior is broader than drag/drop. Shop purchases, world pickups, rewards, mailbox claims and quick Storage withdrawals should eventually reuse the same authoritative Inventory placement rules. They do not need to go through ItemDragRuntime.

```
receive_item(item, optional_requested_position)

requested position
    -> require exact fit

no requested position
    -> find first fitting position
```

10. Window policies

NPC Shop

NPC Shop offers are draggable representations, not owned items. Dragging an NPC Shop offer to Inventory means begin purchase flow, not move item. Right-clicking the same offer should enter the same workflow.

```
right click --              -> quantity / confirmation -> shop.buy
drag/drop  --/
```

NPC stock may be infinite. The runtime never assumes the source item disappears.

Player Shop

Buyer mode behaves similarly to NPC Shop: dropping a Player Shop offer onto Inventory begins a purchase workflow. Owner/editor mode uses different policies: Inventory to Player Shop begins a listing workflow; own listing to Inventory removes the listing.

The same visual shop can therefore register different drag capabilities depending on whether the current player is the owner or a buyer.

Trade

The player's own trade area can be both source and target. Inventory to own offer means trade.add_item. Own offer to Inventory means trade.remove_item. The opponent's offer is presentation-only and cannot be dragged. Once a trade side becomes locked or accepted, its drag capability can be disabled without changing ItemDragRuntime.

Equipment

Equipment slots are non-grid targets. Inventory sword to weapon slot means equipment.equip. Invalid item types produce a rejected preview. Direct Storage-to-Equipment interaction is not required for the initial implementation; items should first enter Inventory.

11. Purchase capacity and atomicity

Purchasing must never charge the player and then discover that Inventory is full. The World server must validate destination capacity as part of the authoritative purchase operation.

NPC purchase must validate:

- offer exists

- item definition exists
- quantity is valid
- player has enough currency
- requested/free Inventory placement exists

before atomically committing:

- currency deduction
- item creation
- Inventory placement
- NPC stock update, if applicable

Player Shop purchase must validate:

- listing still exists
- seller still owns or controls the listing
- buyer has enough currency
- buyer has valid Inventory placement

before atomically committing:

- buyer currency deduction
- seller payment
- item ownership transfer
- buyer Inventory placement
- listing removal or quantity reduction

Failure to find space must return a stable error such as inventory_full without charging currency or consuming the offer. Client-side free-space checks may improve UX, but are never authoritative.

12. Ownership invariant

ItemDragRuntime never changes ownership. Ownership changes only through authoritative domain operations.

Interaction	Ownership result
Inventory -> Inventory	Ownership unchanged
Storage -> another character on same account	Ownership assigned by Storage domain
NPC Shop -> Inventory	New ItemInstance created by server
Player Shop -> Inventory	Ownership transferred by Shop domain
Trade completion	Ownership transferred by Trade domain

13. Preview invariant

A green preview means: the client currently believes this interaction is locally valid. It never guarantees server acceptance.

- stale revision
- changed Inventory occupancy
- item ownership change
- offer sold

- trade locked
- insufficient currency
- inventory full
- server-side eligibility rules

The authoritative snapshot always wins.

14. Initial implementation scope

ItemDragRuntime v1 should migrate existing behavior for Inventory, Storage and Equipment only.

- Inventory -> Inventory movement
- Inventory <-> Storage
- Storage -> Storage
- Inventory -> Equipment
- Equipment -> Inventory
- drag-to-carry
- click-to-carry
- page switching while carrying where currently supported
- green/red placement preview
- Escape cancellation
- right-click cancellation
- UI scaling
- server rejection handling
- Ctrl-click quick transfer
- item.activate right-click behavior

NPC Shop, Player Shop and Trade are design acceptance cases for the API, but are not part of the first migration.